



Module 6.1

Combinatorial Building Blocks

cādence®

This page does not contain notes.

Module Objective

In this module, you will:

- Construct simple combinational logic.

Topics include:

- Basic gates
- Multiplexors
- Decoders
- Encoders
- Priority encoders



In this module, we will construct simple unclocked, combinational logic using basic gates. We will look at multiplexors, decoders and encoders.

Basic Gates: Inverter

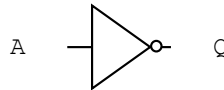
All combinational logic is constructed from simple gates:

- These are constructed from simple transistor circuits.
 - Technology dependent.

The simplest is the Inverter (NOT).

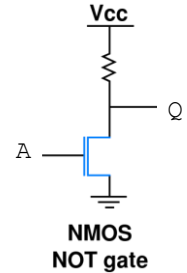
- Arithmetic symbol
 - Used for defining and manipulating logic with arithmetical equations (boolean algebra).
- Schematic symbol
 - Used for defining logic with diagrams (schematics).
- Truth Table
 - Used for defining gate or logic functionality.

$$Q = \overline{A}$$



A	Q
0	1
1	0

Transistor Implementation



All combinational logic is constructed from simple logic gates, which are themselves constructed from simple transistor circuits. The exact implementation depends on the transistor technology. The simplest gate is the inverter implementing the NOT operator. This simply inverts the input, so a logic 0 becomes a logic 1 and vice-versa. There are various representations of an inverter for different purposes. The arithmetic symbol is used in defining and manipulating binary arithmetic (called Boolean algebra) and uses an over score or overline. The schematic symbol is used in graphical circuit diagrams, called schematics, and a truth table defines the behavior of the gate. A simple NMOS technology implementation uses a single transistor for an inverter. For a logic 1 input, the transistor turns on and connects the output to ground. For a logic 0 input, the transistor turns off, and the resistor pulls the output to high.

AND and NAND Gates

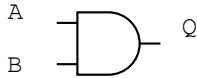
AND

Multiplication of bits

- Arithmetic symbol

$$Q = A \cdot B$$

- Schematic symbol



- Truth table

AND

A	B	Q
0	0	0
0	1	0
1	0	0
1	1	1

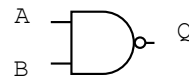
NAND

AND with inverter

- Arithmetic symbol

$$Q = \overline{A \cdot B}$$

- Schematic symbol



- Truth table

NAND

A	B	Q
0	0	1
0	1	1
1	0	1
1	1	0

The AND gate is the multiplication of inputs. If A and B are 1, then the output is 1, else 0. The arithmetic symbol is a dot and the slide shows the schematic and truth table. The output of AND can be inverted to form a single NAND (Not-AND) gate, which has the symbols and truth table as shown. A circle on the output of the schematic symbol indicates the inversion.

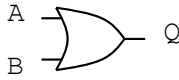


OR, NOR, XOR, and XNOR Gates

OR

- Addition of bits

$$Q = A + B$$

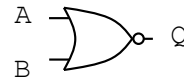


OR		
A	B	Q
0	0	0
0	1	1
1	0	1
1	1	1

NOR

- OR with inverter

$$Q = \overline{A + B}$$

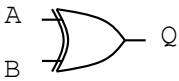


NOR		
A	B	Q
0	0	1
0	1	0
1	0	0
1	1	0

XOR

- Exclusive OR
 - Output 1 if inputs are different

$$Q = A \oplus B$$

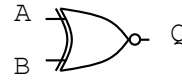


XOR		
A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0

XNOR

- XOR with inverter

$$Q = \overline{A \oplus B}$$



XNOR		
A	B	Q
0	0	1
0	1	0
1	0	0
1	1	1

5

© Cadence Design Systems, Inc. All rights reserved.

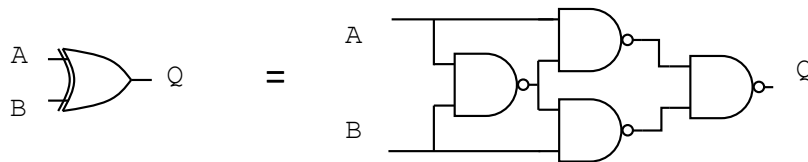
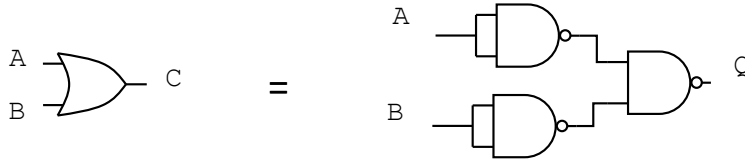
cadence

The OR gate is addition of inputs. If either A or B are 1, then the output is 1 else 0. The arithmetic symbol is a plus sign, and the schematic and truth table are as shown. We add an inverter to an OR gate to form NOR. There is variant of OR called XOR or exclusive OR. Here the output is 1 if the inputs are different. The symbol is a plus enclosed in a circle. The output of XOR can be inverted to form an XNOR.

Functional Completeness

NAND and NOR gates are functionally complete.

- All other logic can be built with either NAND or NOR.
- Entire processors can be built just with either gate.



All combinational logic is built from the simple gates we have shown. However, real-life hardware implementations are even simpler than this. They rely on the concept that NAND and NOR gates are functionally complete. This means that all logic gates can be built with combinations of either NAND or NOR. For example, an inverter is a NAND gate with both inputs connected together. An OR gate is two NAND invertors followed by a NAND. So, hardware can be implemented with a vast array of identical NAND gates, which are then connected together to give the required logic. In fact, entire processors can be built with just NAND or NOR gates.

Multiplexor

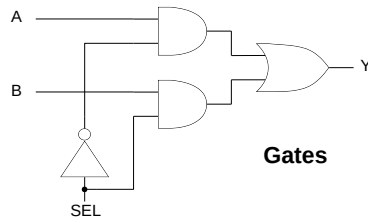
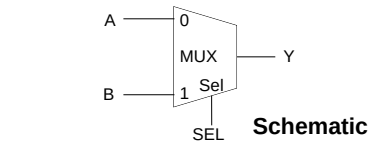
Multiplexor selects A or B input based on the value of SEL.

X is don't care
(could be 1 or 0)

A	B	SEL	Y
p	X	0	p
X	q	1	q

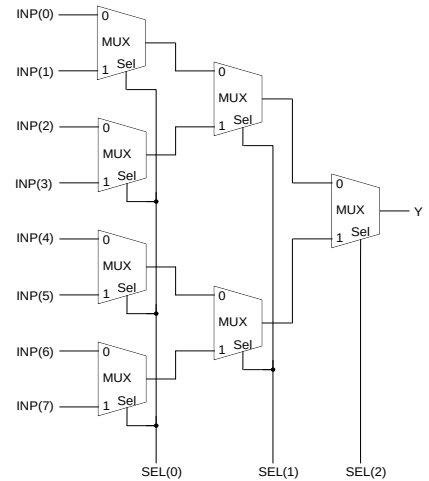
Propagation delay

Truth Table



$$Y = \overline{A} \cdot \text{SEL} + B \cdot \text{SEL}$$

Arithmetic



MUXes can be scaled for any size input.

cadence

MUX connects the 0 input to Y when SEL is 0 and the 1 input to Y when SEL is 1.

The gate level AND-OR circuit points to a specific implementation, but a multiplexer can be constructed in a number of different ways.

Note: MUX can be used to implement logical gates by various connections:

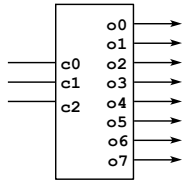
- Connect A and SEL to give AND gate

Note that building the 8 input MUX from 2-input MUXes results in a logical tree-like structure. Again, this is only one way of implementing the multiplexer.

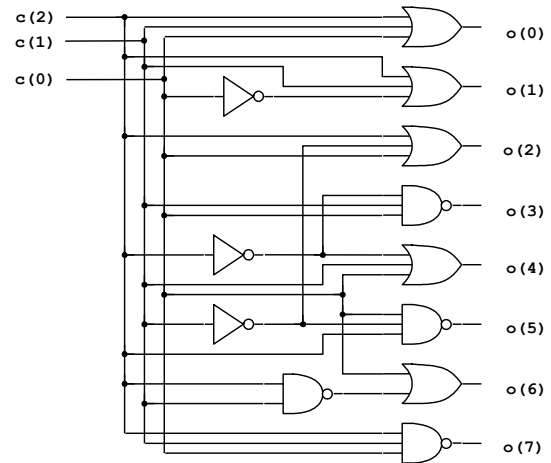
We don't describe designs in terms of basic gates; we use larger blocks, and here is one example, the multiplexor or MUX. A simple MUX uses the value of the input SEL to pass either the A or B data input to the output Y. The implementation is simple, as you can see from the gate diagram, although a MUX can be implemented in many ways. Note that the truth table uses additional symbols, as well as 1 and 0, for simplicity. For example, when SEL is 0, input A is selected, and the value of A (p is either 1 or 0) is output on Y. As B is not selected, we don't care about its value, indicated by X. There is also a time delay between the inputs changing and output updating, as the logic values propagate through the multiplexor gates. A MUX can be scaled for any size of the input. This example shows an 8 to 1 MUX with a 3-bit selector, implemented in 3 stages of 2 input MUXes. If SEL is 4 (binary 100), INP(4) is selected.

Decoder Architecture

Decoder usually selects one output based on the value of the input.



c2	c1	c0	o7	o6	o5	o4	o3	o2	o1	o0
0	0	0	1	1	1	1	1	1	1	0
0	0	1	1	1	1	1	1	1	0	1
0	1	0	1	1	1	1	1	0	1	1
0	1	1	1	1	1	1	0	1	1	1
1	0	0	1	1	1	0	1	1	1	1
1	0	1	1	1	0	1	1	1	1	1
1	1	0	1	0	1	1	1	1	1	1
1	1	1	0	1	1	1	1	1	1	1



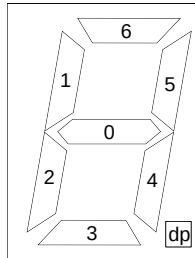
Note that architecture is superficially similar to the multiplexer – however, gate-level implementation is very different.

The example truth table converts a binary representation into a “one-cold” representation.

Another example of a combinatorial building block is a decoder. A decoder typically has more output than input bits and directly processes the inputs to produce the output. Although the architecture is superficially similar to the MUX, the gate-level implementation is very different. This example converts a binary input c to clear a single output bit of o , i.e., if c is 3, o_3 is zero, and all other bits are one. This binary representation, where only one bit can be zero, and all others are 1, is known as one-cold. The inverse, where only 1 bit can be 1, and all others are 0, is called one-hot. There are further variants; for example, “one-hot or all-cold” allows all bits to be 0 as well as 1 bit to be 1.

7-Segment Decoder Example

For viewing on a 7-segment LED display, a number must be translated into a pattern of lit segments.



Values >9 undefined

Relationship depends on display

input	o7	o6	o5	o4	o3	o2	o1	o0
0000	1	1	1	1	1	1	1	0
0001	0	0	1	1	0	0	0	0
0010	1	1	1	0	1	1	0	1
0011	1	1	1	1	1	0	0	1
0100	1	1	1	1	0	0	1	1
0101	1	1	0	1	1	0	1	1
0110	1	1	0	1	1	1	1	1
0111	0	1	1	1	0	0	0	0
1000	1	1	1	1	1	1	1	1
1001	1	1	1	1	0	0	1	1
1010...	undefined							
1111								

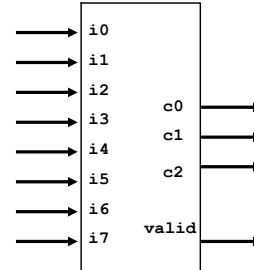
This example decodes a 4-bit input to be displayed in decimal on a 7-segment LED display. The relationship depends on the display, but here to display a 2 requires segments 0,2,3,5 and 6 to be lit. An input of 8 lights all segments. Note the output for binary values greater than 9 is not defined as these cannot be displayed. We may choose to show an E for error by setting all segments except 4 and 5.

Encoder

Encoder usually outputs value based on input value or a combination of inputs.

- E.g., one-hot to binary with valid.

i7	i6	i5	i4	i3	i2	i1	i0	c2	c1	c0	valid
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	0	1	0	0	0	1	1
0	0	0	0	0	1	0	0	0	1	0	1
0	0	0	0	1	0	0	0	0	1	1	1
0	0	0	1	0	0	0	0	1	0	0	1
0	0	1	0	0	0	0	0	1	0	1	1
0	1	0	0	0	0	0	0	1	1	0	1
1	0	0	0	0	0	0	0	1	1	1	1



The opposite of a decoder is an encoder, which typically has more inputs than outputs. This encoder has a "one-hot or all-cold" input pattern and encodes the one-hot bit as a binary output. It also has an additional valid output which is cleared for the all-zero input, but set otherwise, allowing us to distinguish between the i0 one-hot and the all-cold inputs.

Priority Encoder

Inputs have a priority.

- Encoder returns a value corresponding to the highest priority input that is set.

pri7	pri6	pri5	pri4	pri3	pri2	pri1	pri0	c2	c1	c0	valid
0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	0	1	X	0	0	1	1
0	0	0	0	0	1	X	X	0	1	0	1
0	0	0	0	1	X	X	X	0	1	1	1
0	0	0	1	X	X	X	X	1	0	0	1
0	0	1	X	X	X	X	X	1	0	1	1
0	1	X	X	X	X	X	X	1	1	0	1
1	X	X	X	X	X	X	X	1	1	1	1

X is don't care
(could be 1 or 0)



Unlike previous cases, the priority encoder is more naturally expressed using if..then..else, because it has the element of prioritization.

Classic use of a priority encoder is to change the output of a chain of comparators in a flash analog to digital converter into a binary word. A 10-bit A/D requires a 1024 to 10-bit priority encoder.

A priority encoder is an encoder where the input bits have a defined priority. Here, higher inputs bits have priority over lower bits. We use X in the truth table to indicate don't care where the bit could be 1 or 0. So if pri7 is one, we don't care about the other inputs, the output is 111 and valid is set. If pri7 is 0, then we check the next highest priority bit pri6. If set, the other bits are ignored, and the output is 110. The lowest priority bit is pri0, which is only checked if all other bits are zero. Finally, an all-cold input outputs zero, but with valid cleared. Therefore, the output corresponds to the highest priority bit set in the input.

Summary

In this module, you learned:

All combinatorial logic is built from standard blocks:

- Which, in turn, are constructed from basic gates.
 - For which we can exclusively use functional complete gates like NAND or NOR.
- MUXes conditionally select between different data inputs depending on select control inputs.
- Decoders typically convert a small number of inputs into a larger number of outputs.
- Encoders typically convert a large number of inputs into a smaller number of outputs.
- Considering input priority for a priority encoder.



In summary, all combinatorial logic is built from standard blocks, which are, in turn, are constructed from basic logic gates. The hardware implementation can use exclusively NAND or OR gates as these are functionally complete. MUXes select between data inputs depending on a select control input. Decoders typically have more outputs than inputs, and encoders have more inputs than outputs. If the inputs of an encoder are prioritized, then this is a priority encoder.



Module 6.2

Clocked Building Blocks and Synchronous Design

cādence®

This page does not contain notes.

Objective

In this module, you will:

- Identify logic storage devices and their use.

Topics include:

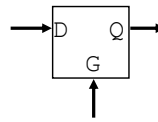
- Latches and flip-flops
- Latch versus flip-flop based design
- Combinational and synchronous timing
 - Set up and hold time
- Clock delays
- Effective clock periods



This submodule will look at storage devices. We will compare latches and flip-flops. We will look at combinational and synchronous timing and explore setup and hold times for registers. We will also examine clock delays and effective clock cycles.

Latch

D	G	Q
X	0	<i>Previous value</i>
0	1	0
1	1	1

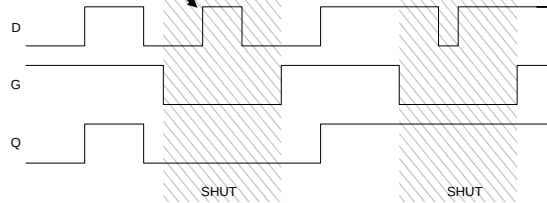


Stores the state of the input when there is a 0 on the gate input.

level-sensitive

Output “latched”

Output open

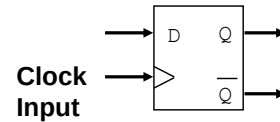
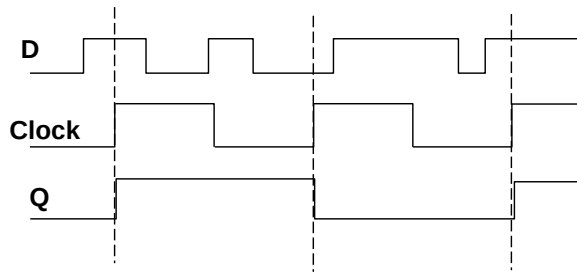


The basic storage element is the latch. This has a gate input G, and if the gate is high, then changes on the input D are passed to the output Q, and the latch is open. When G goes low, then the latch is shut, and the last value on D is “latched,” stored internally and driven on Q. While the latch is shut, changes on D are ignored. The latch is level sensitive in that the latch is open while G is high and shut while G is low.

Flip-Flop

Most commonly used flip-flop is the D-type.

- Stores the state of the input when there is a transition of the clock input.
- Edge-triggered.



D_n	Q_{n+1}	$\overline{Q}_{n+1}$
0	0	1
1	1	0

Positive Edge Triggered FF

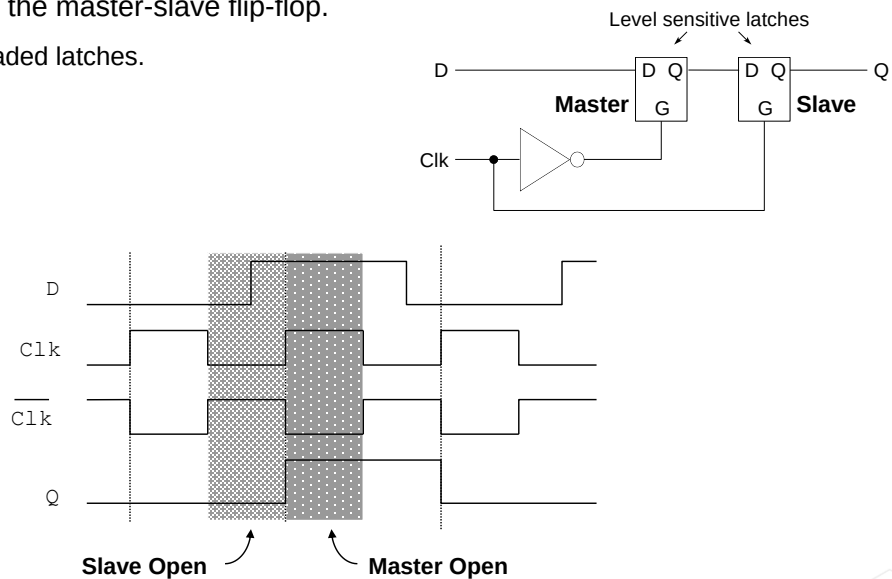


However, latches are rarely used. The most common storage device is the flip-flop, of which the most common variant is the D-type flip-flop. This has a clock input, and the flip-flop samples the input on a clock transition, stores the value internally and drives the value on the output Q and the inverse on the Q bar. The flip-flop is edge-triggered in that a clock transition samples the input rather than a clock level. Flip-flops can be clocked on either the positive edge of the clock (0 to 1 transition) as here, the negative edge, or in some cases, both edges. Flip-flops clocked on both edges are used in Double-Data Rate (DDR) designs, typically where high data rates are required, such as memory access.

Flip-Flop Implementation

One implementation is the master-slave flip-flop.

- Consists of two cascaded latches.



17 © Cadence Design Systems, Inc. All rights reserved.



A flip-flop can be implemented as a master-slave with two cascaded latches. The clock is inverted to gate the first master latch, which is open while the clock is low. The second slave latch is gated directly from the clock and is open while the clock is high. While the clock is low, the master is open, and the slave holds the value of the output Q. When the clock goes high, the master shuts, the slave opens, and the input is sampled and driven on the output Q by the master. When the clock goes low, the master opens, the slave shuts, and the output of the master is sampled and driven on the output Q by the slave.

Flip-Flop or Latch-Based Design?

Flip-flops are convenient

- Described easily
- Fewer timing issues
- Easier test implementation
- Expensive in terms of area

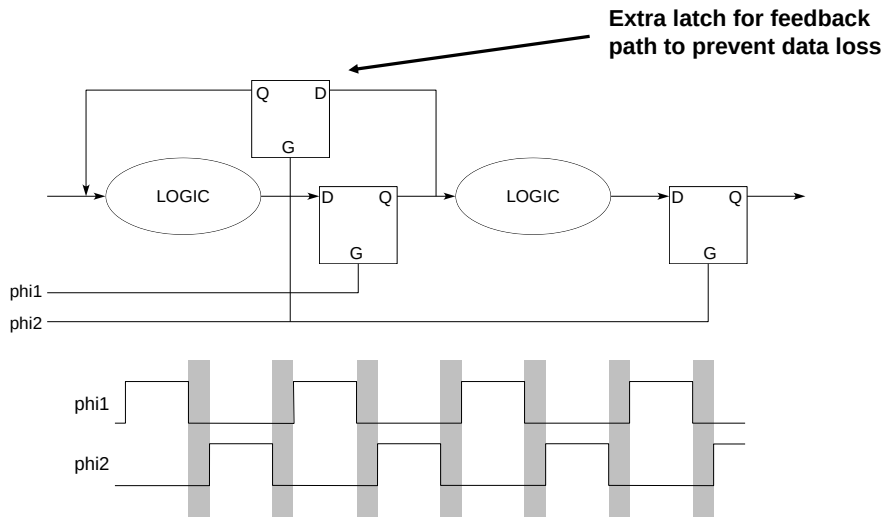
Greater efficiency can be achieved using latches

- Area savings of up to 50%
- Need 2-phase non-overlapping clocks
- Design is more difficult
- Test implementation more difficult



Although designs can be either flip-flop or latch-based, flip-flop designs are standard. They can be described more easily, are implemented with fewer timing issues, and are more easily tested. However, flip-flop designs are up to twice as large as latched designs in terms of area. Latch-based designs are more difficult to design as they need 2-phase, non-overlapping clocks, and testing is more difficult.

Latch-Based Design



19 © Cadence Design Systems, Inc. All rights reserved.



Here is the clocking scheme for a latch-based design. It requires two clocks, phi1 and phi2, which are of opposite phases and non-overlapping. The clocks are connected to alternate latches so that when phi1 is high, the two logic blocks are connected, but data is blocked by the latch clocked by phi2. The phi1 latch shuts before the phi2 latch opens, so data is passed sequentially through the logic blocks by the alternate latches. Note that if a logic block requires a feedback path, we need an extra latch gated from the other clock to hold the feedback data and prevent an open loop.

Latch Design Issues

Latches are “open” for part of the time.

- Flip-flops transfer data at edges.
- If the logic before the storage element undergoes spurious transitions, an open latch will allow these to propagate.

This may:

- Cause problems in later parts of the circuit.
- Increase power consumption.



Another issue with latch designs is that latches are open for part of the time. Any transitions on the logic before the latch will be propagated to the next logic block. This could cause problems in logic downstream from the latch. The transitions also increase power consumption. Flip-flop designs sample data at the rising or falling edge, which filters out spurious data transitions and gives us a more stable design with lower power consumption.

Combinational Timing

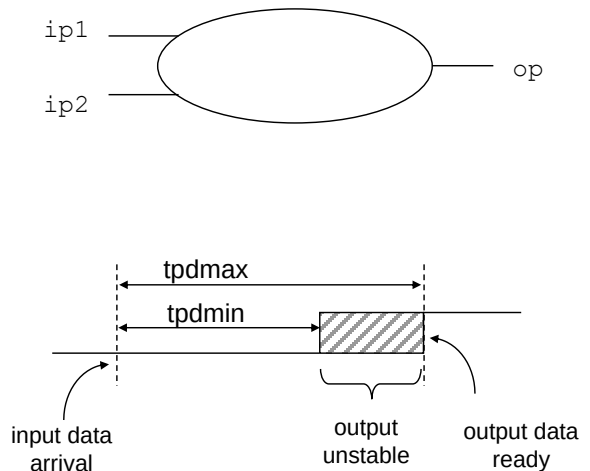
Each gate in combinational logic has a delay:

- Propagation delay t_{pd}

Typically, many logic paths from inputs to outputs:

- Shortest path is minimum delay t_{pdmin}
- Longest path is maximum delay t_{pdmax}
- Between t_{pdmin} and t_{pdmax} , outputs are changing

$t_{pdmax} = 20\text{ns}$
 $t_{pdmin} = 15\text{ns}$
delay input to output = 20 ns
latency
new data input every 20 ns
throughput



Let's talk about timing and, firstly combinational timing. Each basic gate has a propagation delay t_{pd} between the inputs changing and the outputs updating. The more gates in a logic path from input to output, the greater the delay. However, there are typically many logic paths through a combinational block. The shortest path gives the minimum delay t_{pdmin} , and the longest path the maximum t_{pdmax} . Therefore, from data arriving at the input to a block, the outputs will be changing between t_{pdmin} and t_{pdmax} after the arrival time. The outputs will not be ready until after t_{pdmax} . The maximum delay from input to output, t_{pdmax} , is called the latency. If we sample output data as soon as it is ready, then new data can arrive at a minimum of t_{pdmax} after the previous data, and this gives us the throughput of the block.

Register Timing: Setup, Hold and Clock-to-Q

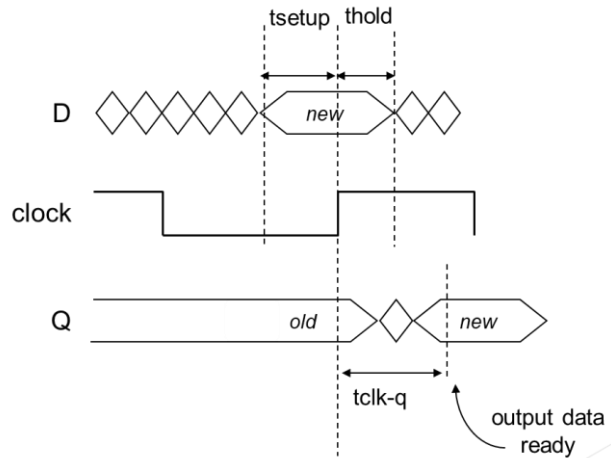
A register samples D and outputs Q on the clock edge.

Sampling requires that the input remains stable for:

- Setup time before the clock rising edge (t_{setup}).
- Hold time after the clock rising edge (t_{hold}).
- If setup or hold times are violated, the register samples a changing value.
 - No guarantee of value stored.
 - Metastability problems.

Output appears on Q as a short delay after the clock edge.

- Clock to Q delay (t_{clk-q}).

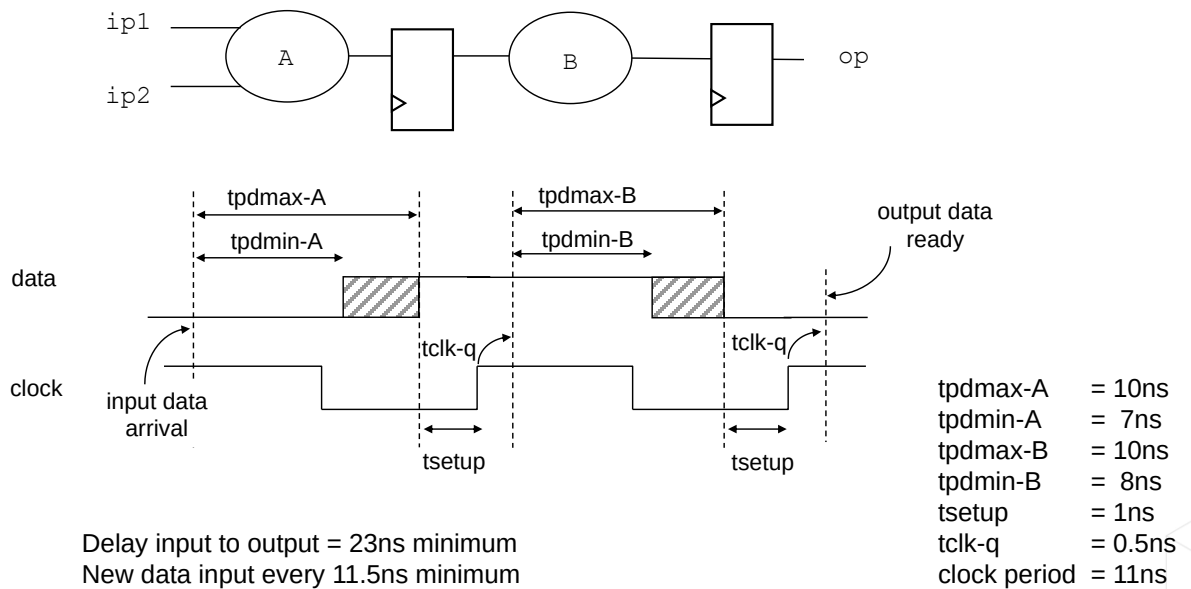


If setup and hold times are violated, then the register is trying to sample the input D while it is changing. Sampling a changing value can lead to either an indeterminate value being sampled (either 0 or 1, but we don't know which) or meta-stability. In metastability, the input D value sampled may be between the voltage thresholds for logic 0 or logic 1. This may result in the transistors of the register no longer behaving as switches but turning partially on. The metastability condition can then be passed to subsequent logic in the design. Metastable conditions on the register output may not last long but could last long enough to cause problems.

Metastability can be a specific issue for asynchronous inputs, which are not synchronized to the clock and, therefore, may violate the setup and hold times. There are standard techniques for handling such inputs, such as double-registering. This uses two registers in series on the input, with the aim that if the first register has metastability, there is a full clock cycle for the output to stabilize before being sampled by the second register.

Flip-flops, or registers, have their own timing constraints. The input D is sampled on the rising clock edge, but registers require D to be stable for a time before the edge called the setup time (t_{setup}). D must also be stable for a time after the clock edge, called the hold time (t_{hold}). If either setup or hold time is violated, the register samples and outputs a changing value which may be between the logic voltage thresholds. This is called metastability. An intermediate voltage between thresholds can result in the transistors of a gate no longer behaving as switches but partially turning on and propagating the value to other logic in the design. A metastable condition may not last long, as the voltage is dragged to one of the logic levels by noise, crosstalk, or other electronic effects in the hardware, but the condition may last long enough to cause spurious transitions and other problems. The final timing characteristic of registers to consider is clock to Q delay t_{clk-q} . The output Q changes a short delay after the clock edge.

Sequential Timing



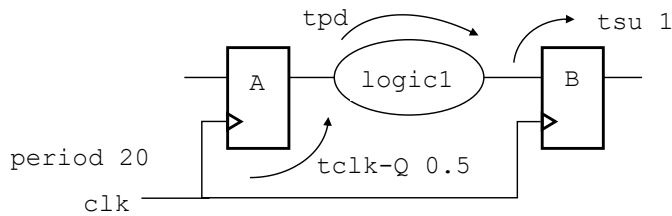
23 © Cadence Design Systems, Inc. All rights reserved.

cadence

Putting together combinational propagation delay and register timing allows us to start to build a timing model for the design.

By putting together combinational and register timing, we can analyze design timing. Tpdmax for logic A is 10ns, and the setup time for the register is 1ns, so new data could arrive at A with a minimum of 11ns apart. Data is released from the register after 0.5ns (tclk-q), the maximum delay through B is 10ns (tpdmax-B), and the data must arrive 1 ns before the clock edge (tsetup). Therefore, the minimum clock period for logic B is 11.5ns. The delay from input to output is tpdmax-A + tsetup + tclk-q + tpdmax-B + tsetup + tclk-q (for the output register) = 23ns.

Setup Time



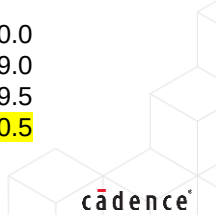
tpdmax = 18

- clk rising edge(1): 0.0
- data op from A: 0.5
- data op from logic1: 18.5
- clk rising edge(2): 20.0
- data expected at B: 19.0
- data arrival at B: 18.5
- slack: +0.5

tpdmax = 19

- clk rising edge(1): 0.0
- data op from A: 0.5
- data op from logic1: 19.5
- clk rising edge(2): 20.0
- data expected at B: 19.0
- data arrival at B: 19.5
- **setup time violation: -0.5**

24 © Cadence Design Systems, Inc. All rights reserved.



If tpd, the *longest* propagation delay through logic1, is 18ns, then:

- A rising edge of the clock arrives at the clock pin of register A at, say, 0ns.
- This releases data on the Q output of register A at 0.5ns (clock -Q delay).
- Data arrives at the input of register B at 18.5ns (18ns propagation delay in logic1).
- The next rising edge of the clock arrives at the clock pin of register B at 20ns.
- The setup time sets the latest possible time for the data to be ready on the input of B at 19ns (clock arrival – setup time).
- Data is present at 18.5ns, must arrive by 19ns, giving 0.5ns of slack in the timing path.

If tpd, the *longest* propagation delay through logic1, is 19ns, then:

- The data arrives at the input of register B at 19.5ns.
 - Tclk-Q + propagation delay in logic1.
- The next rising edge of the clock arrives at the clock pin of register B at 20ns.
- The setup time sets the latest possible time for the data to be ready on the input of B at 19ns (clock arrival – setup time)
- Data is present at 19.5ns but must arrive by 19ns, giving a setup timing violation of 0.5ns in the timing path.

The clock period and register setup time restrict the maximum propagation delay of logic. Consider this design with a clock period of 20ns, tsetup 1ns and tclk-q 0.5ns. If the tpdmax of logic1 is 18 ns and a clock-rising edge arrives at register A at, say, 0ns. Data is output from A at 0.5ns (tclk-q) and arrives at register B at 18.5ns. The next clock edge arrives at B at 20ns. Tsetup is 1ns, so the latest the data can arrive at B is 19ns (clock period – tsetup). If the data is present at 18.5ns, that gives 0.5ns of slack in the timing path. If, however, logic1 tpdmax is 19ns, then the data arrives at register B at 19.5ns. This is 0.5ns into the setup time, so there is a 0.5ns timing violation on B.

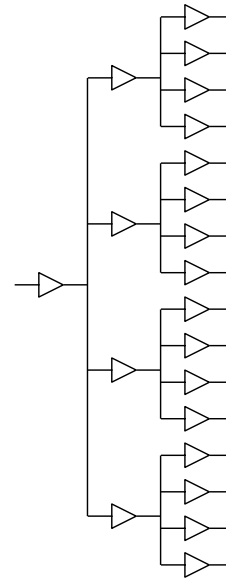
Clock Distribution

Clock distribution is very important:

- A single clock may route to every register in the design.
- Long, heavily loaded nets.
- Major delay and power considerations.

Special techniques for clock distribution:

- FPGA: dedicated fast global routing.
- ASIC: clock tree inserted during layout.
 - Must be “load balanced.”
 - Every branch sees the same load (approx.).
 - To minimize propagation delay, transition time, and, therefore, power.
 - Special tools or manual tweaking.
 - Several architectures.

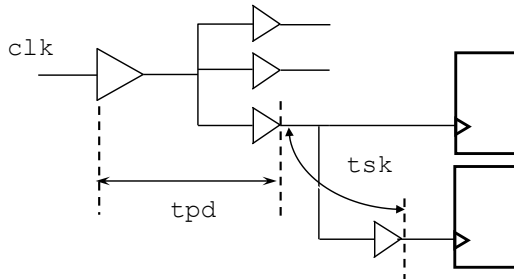


25 © Cadence Design Systems, Inc. All rights reserved.



Before we look at hold time, we need to address clock distribution. Every register in the design could be connected to a single clock, so clock nets tend to be very long and heavily loaded. As length and load contribute to power and delay, there are special techniques for optimizing clock distribution. Field-Programmable Gate Arrays (FPGAs) have fast, dedicated global connections built into the technology, but for ASIC devices, the clock connections must be inserted during layout. The aim is to balance the clock tree so every branch sees about the same register load and to minimize propagation delay transition time and power. There are several architectures that can be used, and special software tools, sometimes aided by manual tweaking, to implement these architectures.

Modeling Clock Trees



Two parameters to model:

- tpd: propagation delay through the clock tree.
- tsk: skew between branches.

Positive skew affects hold timing.

Negative skew affects setup timing.

There are two parameters to consider when modeling clock trees. First is the propagation delay between the clock input and the arrival at a register. Second, is the skew, which is the difference between clock propagation for adjacent registers. Positive skew indicates the clock arrives at a source register before the target register. This affects hold timing. In negative skew, the clock arrives at a target register before the source register, and this affects setup timing.

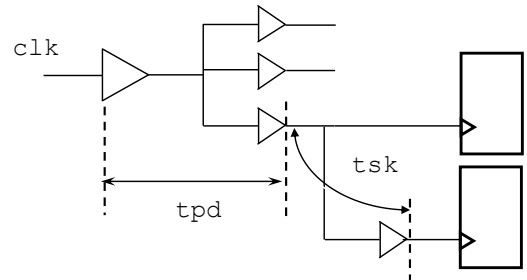
Modeling Clock Trees

Two parameters to model:

- tpd: propagation delay through the clock tree.
- tsk: skew between branches.

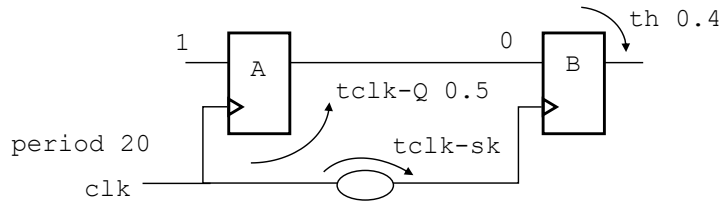
Positive skew affects hold timing.

Negative skew affects setup timing.



There are two parameters to consider when modeling clock trees. First is the propagation delay between the clock input and the arrival at a register. Second, is the skew, which is the difference between clock propagation for adjacent registers. Positive skew indicates the clock arrives at a source register before the target register. This affects hold timing. In negative skew, the clock arrives at a target register before the source register, and this affects setup timing.

Hold Time



$t_{clk-sk} = 0$

- clk rising edge(1) at A: 0.0
- data op from A: 0.5
- data arrival at B: 0.5
- clk rising edge(1) at B: 0.0
- data expected at B: 0.4
- data arrival at B: 0.5
- slack +0.1

$t_{clk-sk} = 0.2$

- clk rising edge(1) at A: 0.0
- data op from A: 0.5
- data arrival at B: 0.5
- clk rising edge(1) at B: 0.2
- data expected at B: 0.6
- data arrival at B: 0.5
- **hold time violation -0.1**

Hold time violations are not so common, but a typical example may be in the timing of a shift register where the clock net is subject to positive skew, i.e., there is no logic between the registers, and the clock edge arrives at the source register *before* the target register.

If clock skew (t_{clk-sk}) is 0ns:

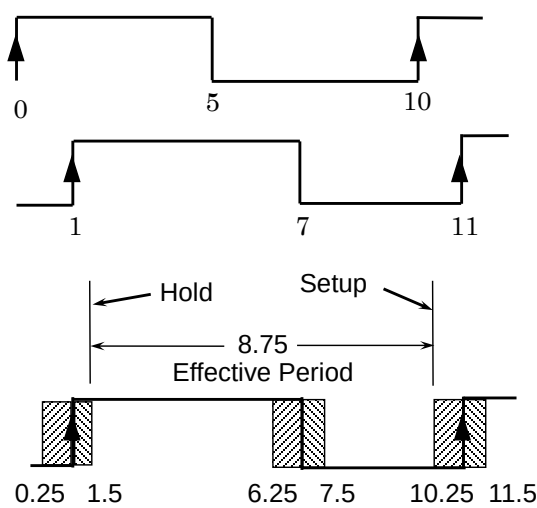
- A clock edge arrives at register A at, say, 0ns and releases a '1' onto the output.
- The '1' arrives at the input of register B at 0.5ns (t_{clk-q}).
- The clock edge arrives at register B at 0ns.
- This means the data '0' currently at the input of B must remain unchanged until 0.4ns.
 - Clock arrival + hold time.
- The '0' must remain until 0.4ns is overwritten at 0.5ns, giving 0.1ns of slack in the timing path.

If the clock skew (t_{clk-sk}) is 0.2ns:

- The '1' arrives at the input of register B at 0.5ns (t_{clk-q}).
- The **same** clock edge arrives at register B at 0.2ns due to clock net delay.
- The data '0' currently at the input of B must remain unchanged until 0.6ns.
 - Clock arrival + hold time.
- The '0' must remain until 0.6ns, but is overwritten at 0.5ns, giving a hold time violation of 0.1ns in the timing path.

Hold time violations are not common, but a typical example may be for adjacent registers with no combinational logic, where the clock has positive skew and arrives at the source register before the target register. Consider this design with a clock period of 20ns, thold 0.4ns and t_{clk-q} 0.5ns. The input of register A is 1 and B is 0. The clock edge arrives at A at say 0ns and releases a 1 from A at 0.5ns (t_{clk-q}). The 1 arrives at register B at 0.5ns. If the skew t_{clk-sk} is 0ns, then the clock edge arrives at B at the same time as A, 0ns. The current input at B, 0, must remain unchanged until 0.4ns (thold). The 1 from A does not arrive until 0.5ns, so we have a slack of 0.1ns in the timing path. If, however, the clock skew is 0.2ns, the clock edge arrives at B at 0.2ns. The 0 input at B must remain unchanged until 0.6ns (clock arrival + thold) but is overwritten by the 1 arriving from A at 0.5ns, giving a hold time violation of 0.1ns.

Effective Clock Period



10ns period clock

+ve edge delay 1ns

-ve edge delay 2ns

Max. +ve skew 0.5ns

Max. -ve skew 0.75ns

Effective period is 8.75ns

Clock propagation and skew can mean registers see an effective clock period that is different from the period of the input clock. If we have a 10ns period clock rising at 0ns, first, we apply propagation delay to the clock edges. It is common to have different delays for positive and negative edges, and this is technology specific. We take the worst-case scenario of a register with the maximum positive skew, with the latest arriving clock edge, feeding a register with the maximum negative skew, with the earliest arriving edge. The latest the edge arrives at the first register is 1.5ns (edge + propagation delay + maximum positive skew). The earliest the edge arrives at the second register is 10.55ns (edge + propagation delay - maximum negative skew). Therefore, the effective clock period is $10.25 - 1.2 = 8.75$ ns, or 12.5% less than the actual clock period.

Summary

In this module, you learned:

Latches and flip-flops are the basis of most real designs.

- Latches are area efficient but difficult to design.
- Flip-flop designs are simpler but up to 50% bigger.

Synchronous design with a single clock is relatively simple.

- Timing easily understood.
- Implementation straight-forward.
- Synthesis easier.
- Real life is different.
 - Asynchronous inputs.
 - Multiple clock domains.

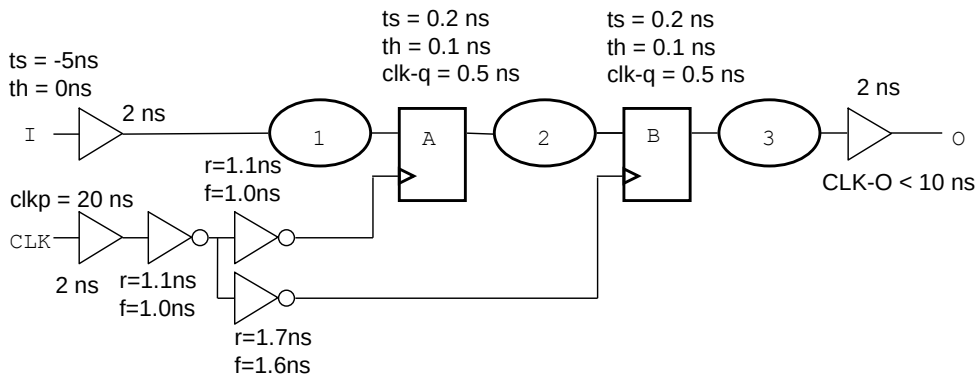
Clock delays must be allowed for in the design.

- Skew and propagation delay can reduce the effective clock period.



In summary, latches and flip-flops storage elements form the basis of most designs. Although latches are more area efficient, latch-based design is more difficult, and so is rarely used. Flip-flops are easier but consume more area. Synchronous designs with a single clock have simple timing and a straightforward implementation, making synthesis much easier. However, real-life designs frequently have asynchronous inputs and multiple clock domains to consider. Remember that clock propagation delay and skew must be allowed for in the design, as they can alter the effective clock period.

Test Your Understanding



- What is the clock propagation and skew at registers A and B?
- What is the effective clock period for logic 2?
- Work out the maximum and minimum delays for clouds 1, 2 and 3.

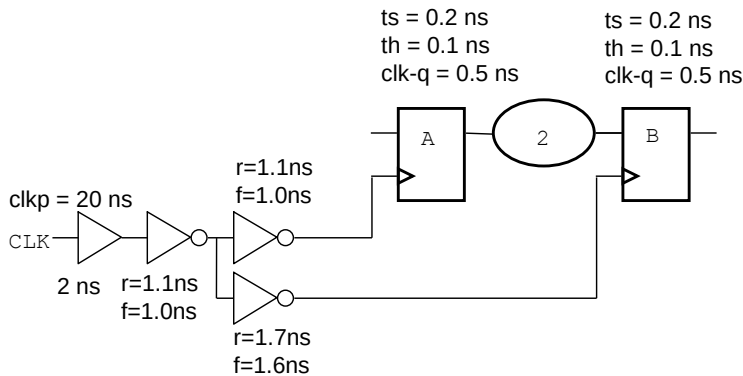
31 © Cadence Design Systems, Inc. All rights reserved.



Assume registers A and B are rising edge triggered.

So, to test what we learned in this module, here is a quick exercise. Pause here, write down your answer, and check the results on the next slides. Note that r is a rising edge delay, and f is a falling edge delay. ts is the setup and th the hold time.

Test Your Understanding (continued)



- What is the clock propagation and skew at registers A and B?
- What is the effective clock period for logic 2?
- Work out the maximum and minimum delays for cloud 2.

32 © Cadence Design Systems, Inc. All rights reserved.



The clock path includes inverters as well as buffers; therefore, when accumulating propagation delays, remember that a rising edge on the input to an inverter becomes a falling edge on the output.

Rising edge clock propagation delay at register A = $2 + 1.1 + 1.0 = 4.1\text{ns}$

Rising edge clock propagation delay at register B = $2 + 1.1 + 1.6 = 4.7\text{ns}$

Therefore, the clock skew at B with respect to A is $+0.6\text{ns}$, and there is no negative skew between A and B.

The effective clock period for logic 2 is, therefore, $20 + 0.6 = 20.6\text{ns}$. This shows that positive skew between two registers can result in an effective clock period greater than the actual clock period.

The maximum delay for cloud 2 relates to setup times. First clock edge arrives at A at 4.1ns and the next clock edge at B at 24.7ns

Data released is from A at 4.6ns . It must arrive at B no later than $24.7\text{ns} - T_{\text{setup}} = 24.5\text{ns}$. Therefore, the maximum delay for 2 is $24.5 - 4.6 = 19.9\text{ns}$.

Minimum delay for cloud 2 relates to hold times. First clock edge arrives at A at 4.1ns , and the same clock edge arrives at B at 4.7ns .

Data released is from A at 4.6ns . It cannot arrive at B earlier than $4.7\text{ns} + T_{\text{hold}} = 4.8\text{ns}$. Therefore, the minimum delay for 2 is $4.8 - 4.6 = 0.2\text{ns}$.

This shows that although positive clock skew can increase the effective clock period, it does so at the expense of a minimum propagation delay between registers. So, there must be some logic in cloud 2, and A and B could be not be part of a shift register.

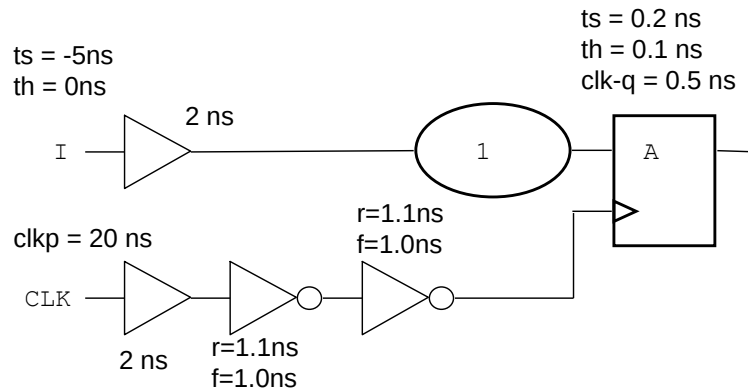
Assume rising edge triggered registers and the CLK rising edge is input at 0ns . The edge arrives at A at $0 + 2 + 1.1$ (rising edge) $+ 1.0$ (falling edge) = 4.1ns . Edge arrival at B is $0 + 2 + 1.1$ (rising) $+ 1.6$ (falling) = 4.7ns . Therefore, with respect to A, the CLK propagation delay is 4.1ns , and the positive skew at B is 0.6ns . Therefore, the effective clock period for logic2 is 20.6ns

Maximum delay for cloud 2. CLK edge arrives at A at 4.1ns . Data released from A at $4.1 + 0.5$ (tclk-q) = 4.6ns .

Next clock edge arrives at B at $20 + 4.7 = 24.7\text{ns}$. Latest arrival for data at B must be $24.7 - 0.2$ (tsetup) = 24.5ns . Therefore, the maximum delay for A is 24.5 (setup at B) $- 4.6$ (data release from A) = 19.9ns .

Minimum delay for cloud 2. Data released from A at 4.6ns . Same CLK edge arrives at B at 4.7ns , and data must be stable until $4.7 + 0.1$ (thold) = 4.8ns . Therefore, the minimum delay for logic2 is 4.8 (hold at B) $- 4.6$ (data release from A) = 0.2ns . The positive skew at B with respect to A gives us an effective clock period greater than the actual clock period at the cost of a minimum delay requirement for cloud 2.

Test Your Understanding (continued)



- Work out the maximum and minimum delays for cloud 1.

The maximum delay for cloud 1 relates to setup times.

Latest data arrival time at the input is 5 ns , and at cloud 1 are, 7 ns .

The next clock edge at A at 24.7 ns , and the setup time means data must arrive at A by $24.7 - 0.2 = 24.5\text{ ns}$. Therefore, the maximum delay for cloud 1 is $24.5 - 7 = 17.5\text{ ns}$.

The minimum delay for cloud 1 relates to hold times.

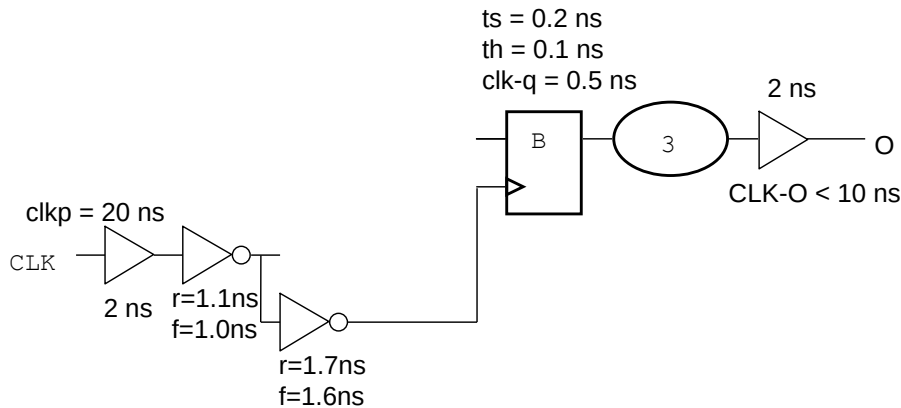
Earliest data can be changed at the input at 0 ns , which changes it at cloud 1 at 2 ns . The first clock arrives at A at 4.1 ns , and so the data cannot change at A before $4.1 + T_{\text{hold}} = 4.2\text{ ns}$.

Therefore, the minimum delay for cloud 1 is 2.2 ns .

This again shows that positive clock skew can increase the effective clock period at the expense of a minimum propagation delay requirement.

CLK edge arrival at A is 4.1 ns . Data arrival at I is -5 ns with respect to the clock, so data arrives at cloud 1 at $-5 + 2 = -3\text{ ns}$. Setup time for A is $4.1 - 0.2$ (t_{setup}) = 3.9 ns . Therefore, the maximum delay for cloud 1 is $3.9 - (-3) = 6.9\text{ ns}$.

Data removal from I is 0 ns , so removed from cloud 1 at 2 ns . Hold time for A is $4.1 + 0.1$ (t_{hold}) = 4.2 ns . Therefore, the minimum delay for cloud 1 is $4.2 - 2 = 2.2\text{ ns}$.

Test Your Understanding (continued)

- Work out the maximum and minimum delays for cloud 3.

The first clock reaches B at 4.7ns, and data is released at 5.2ns.

Clock to output requirement is less than 10ns, so data must be output from cloud 3 at 8ns. Therefore, the maximum delay through cloud 3 is $8 - 5.2 = 2.8$ ns.

As there is no minimum clock-to-output requirement, there is no minimum delay for cloud 3. If cloud 3 were absent, the requirements would still be met.

Output requirement is a maximum delay of 10ns from CLK to O. CLK edge arrives at B at 4.7ns. Data released from B at $4.7 + 0.5$ (tclk-q) = 5.2ns. Data must be released from cloud 3 at $10 - 2 = 8$ ns. Therefore, the maximum delay for cloud 3 is $8 - 5.2 = 2.8$ ns. As there is no minimum delay requirement on the output, there is no minimum delay for cloud 3. So, cloud 3 could be removed, and the specification would still be met.